ACM SIGCITE 2026 · TUTORIAL

Hands-on RAG for Education

From a Generic Chatbot to a Document-Grounded System

Dr. Yong Zheng

Associate Professor

School of Computing, Illinois Institute of Technology

November 13, 2026 · 1:15–2:30 PM · Congdon Hall 1003 · Wilmington, NC

DOI 10.1145/3857770.3858323

Cybersecurity & Information Technology Education

ILLINOIS TECH

OPENING

By the end of this tutorial, you will be able to do five things

RAG = Retrieval-Augmented Generation



Explain how RAG can answer questions from documents, and why this matters for advising



Compare 4 ways to build a RAG assistant by effort, control, and where your documents go



Build an advising assistant from real resources, and score it with a test set



Judge how much you can trust its answers, and design it so that people can check them



Understand additional rules or operations for complicated tasks, e.g., course advising

AGENDA

Today’s 75 minutes include a short opening and four modules



minutes

- **Opening · 5 min** Log in and check that everything works
- **Module 1 · 20 min** Four Ways to Build RAG
- **Module 2 · 35 min** Building an Advising Assistant from Scratch
- **Module 3 · 8 min** How Far Can We Trust the Answers?
- **Module 4 · 7 min** Optional: Course Advising as a More Complicated Task, then Q&A

OPENING

First, log in to the tutorial server and run the first cell of Notebook 01

1

Open the login page

Use the address on the right, or the short URL on the board, or the QR code.

2

Sign in

Choose any username, and use the shared password.

3

Run Section 0 of Notebook 1

Open 01_ready_made_rag.ipynb, click the first code cell, and press Shift + Enter.

Tutorial JupyterHub

Login page

`Announced in the tutorial room`

Username

any name that starts with a lowercase letter and uses only a–z, 0–9, _ and -

Password

`Announced in the tutorial room`

Your turn Log in now and run Section 0. The last line should say `Gateway: OK`.

Instructor Keep this slide up while people arrive, and write the short URL on the board.

OPENING

Everything is on the tutorial website, and it keeps working after class.

Tutorial Website

https://academictutorials.github.io/EdRAG/

Notebooks 1 and 2
versions for the lab server and for Colab

Test questions
expected answers and the scoring rubric

Documents
the original PDF and web page, plus the Markdown versions

Helper code
edrag_utils.py, used by both notebooks

Recorded answers
AnythingLLM answers for the Colab version

Slides
this presentation

Your turn Bookmark the website. After today, use the Colab versions of the notebooks.

Everything is on the tutorial website, and it keeps working after class.

Tutorial Website

https://academictutorials.github.io/EdRAG/

Materials

- Lab resources
  - **Notebooks:** run the lab version on the tutorial JupyterHub during the session, and the Colab version after the tutorial.
    - Notebook 1, three ready-made ways to build a RAG assistant:
      - Lab version (01_ready_made_rag.ipynb)
      - Colab version Open in Colab
    - Notebook 2, build a RAG assistant from scratch:
      - Lab version (02_rag_from_scratch.ipynb)
      - Colab version Open in Colab

- Lab version: what we used today with a running server
- Colab version: adapted resources to be run on Google Colab with Gemini

OPENING

You can run the labs in two places, and the steps and code are the same in both



Tutorial server: the Lab version

Where: JupyterHub on Jetstream2, in your browser

Language model: Qwen3.8-27B from Jetstream2

Backups: Qwen3.8-Flash (ThetaCloud), then Cohere Command A

Embedding model: bge-small-en-v1.5, running on the server



Google Colab: after class, or if the server fails

Where: Your own Google account (a personal Gmail works best)

Language model: Gemini 3.5 Flash-Lite, with your own free Gemini API key

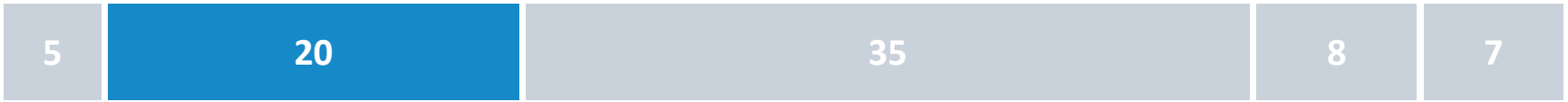
Embedding model: The same bge-small model, running in Colab

How to open: Click the “Open in Colab” buttons on the tutorial website

The two places use different language models, so their answers may not be exactly the same.
The tutorial server will be shut down after class, but you can keep using the Google Colab version to run the code and the tests.

AGENDA

Module 1 (20 minutes): we try three ready-made tools and compare four ways to build RAG



minutes

- Opening · 5 min Log in and check that everything works
- **Module 1 · 20 min Four Ways to Build RAG**
- Module 2 · 35 min Building an Advising Assistant from Scratch
- Module 3 · 8 min How Far Can We Trust the Answers?
- Module 4 · 7 min Optional: Course Advising as a More Complicated Task, then Q&A

Why do we need RAG? Let’s ask a chatbot that has no documents



“I am a student at Illinois Institute of Technology. I am having problems with Canvas. Who should I contact, and how?”

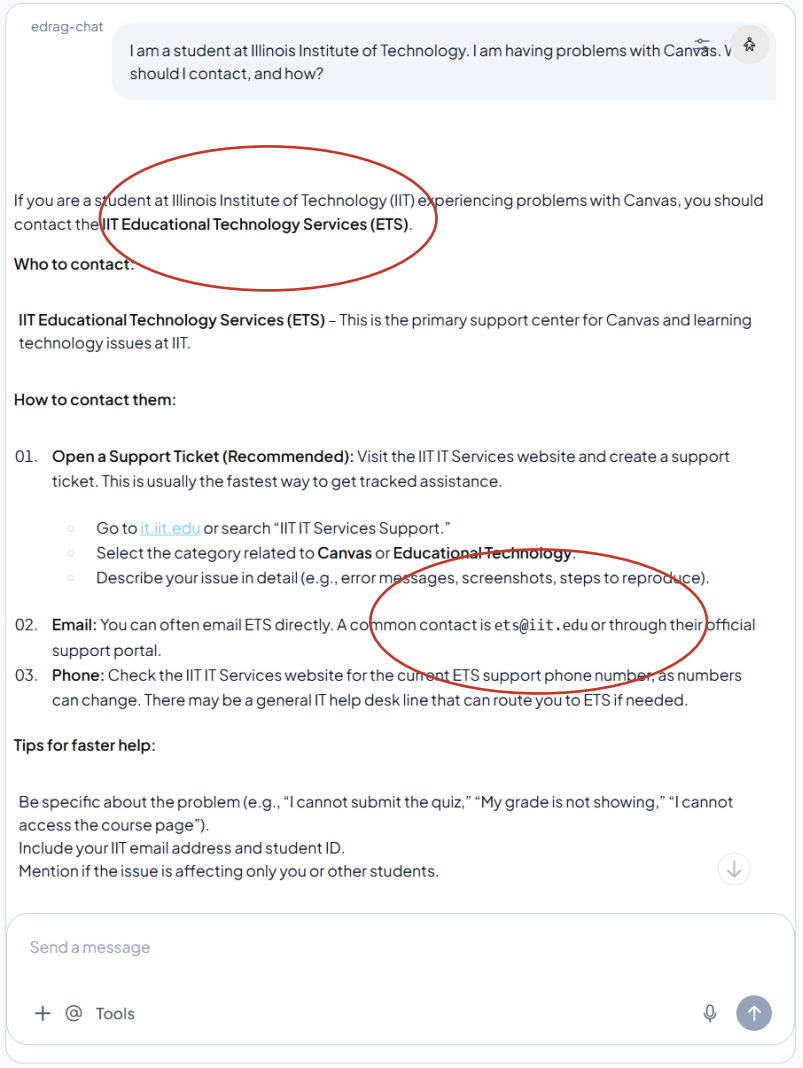
- **It has no access to our documents**
- **It cannot search the web**
- **It is the same model we use later with documents**

Is the contact information in its answer correct?

Your turn Watch the answer. Would you trust it?

Instructor (optional) Chrome tab `localhost:3001` → Tutorial-NoRAG → New Thread → ask the question live

Why do we need RAG? Let’s ask a chatbot that has no documents



Asked without documents, the chatbot named five different help desks, and none was correct

Answer from	Department	Phone	Email
The handbook (correct)	**OTS Support Desk**	**312.567.DESK (3375)**	**supportdesk@illinoistech.edu**
Qwen3.8-Flash, try 1	Academic Information Tech. (AIT)	(312) 567-3400	helpdesk@iit.edu
Qwen3.8-Flash, try 2	Service Desk	(312) 567-3800	servicedesk@iit.edu
Qwen3.8-Flash, try 3	Information Tech. Services (ITS)	(312) 567-3490	none given
Qwen3.8-Max	OTS Support Center	312-567-3400	supportdesk@iit.edu
Cohere Command A	Academic Technology Support	(312) 567-3377	canvas-support@iit.edu

5 different departments

4 different phone numbers

0 correct contact details

Three of the answers told students to ask their instructor, but the handbook says that ITM faculty, teaching assistants, and staff cannot help with Canvas. All the email addresses use the old iit.edu domain.

Recorded before class through the tutorial gateway, with no documents and no web search.

Retrieval-Augmented Generation (RAG) first finds the relevant information pieces, then asks the model to answer from them



Question
A student asks a question.



Retrieve
The system finds the most relevant information pieces (chunks) in our documents.



Build the prompt
The question, the numbered chunks, and some rules go into one prompt.



Generate
The model answers from the chunks and cites them as [1], [2].



Handbook PDF
Tuition web page

Because the answer comes with its chunks, anyone can check each claim against the source.

There are four main ways to build a RAG assistant, from no code to fully custom

	1. Consumer platform	2. Managed RAG API	3. Self-hosted application	4. Built from scratch
Examples	NotebookLM, Claude Projects	Gemini File Search, OpenAI File Search	AnythingLLM, Open WebUI, Dify	Your own code, or LlamaIndex or LangChain
What you need	No code	A few lines of code	No code, but a server to run	Programming
Where documents go	The vendor	The vendor	Your server, but chunks still go to the model	Wherever you decide
Control over retrieval	None	A few settings	Some, depending on the product	Full
Good for	Personal use with a few public documents	Adding Q&A to an existing app	Sensitive documents, for a group with IT support	Teaching, research, and systems you keep testing

Today, we practice on real school documents, in two tasks

Tutorial Website

https://academictutorials.github.io/EdRAG/

- **Task 1: QA chatbots based on RAG from school resources**
  - Tuition rate webpage in html
  - Student handbook in pdf
- **Task 2: Course advising (Optional)**
  - PhD program requirements in html
  - ITM course description in pdf

Our five test questions each check a different way an assistant can go wrong in the QA task

Q1

What is the tuition rate for a graduate student?

Checks whether the assistant reads the right number from a table: $34,650, based on 18 credit hours

Q2

What is the tuition rate for an international student?

Checks whether the assistant admits that the page has no separate international rate, instead of inventing one

Q3

I am having problems with Canvas. Who should I contact, and how?

Checks whether the assistant copies the contact details exactly and keeps the note that ITM staff cannot help

Q4

Can I take two online courses this semester?

Checks whether the assistant gives different answers for F-1 visa students and other students

Q5

May I know Prof. Yong Zheng's office room number and office hours?

Checks whether the assistant says that the documents do not have this information

The documents answer Q1–Q4. They do not answer Q5.



Your turn Keep the expected answers open while you score: `academictutorials.github.io/EdRAG` → Test Questions

We give each answer one score from 0 to 2, and any wrong information means 0

0

Incorrect

The answer contains wrong or invented information, even if the rest is right.

1

Partially correct

Everything it says is correct, but some expected information is missing.

2

Completely correct

Everything it says is correct, and nothing important is missing.

Each approach can earn up to 10 points. After class, you can also score five separate criteria: whether the right chunks were found, whether the answer sticks to them, whether the citations are right, whether anything was made up, and whether it refuses when it should.

There are four main ways to build a RAG assistant, from no code to fully custom

	1. Consumer platform	2. Managed RAG API	3. Self-hosted application	4. Built from scratch
Examples	NotebookLM, Claude Projects	Gemini File Search, OpenAI File Search	AnythingLLM, Open WebUI, Dify	Your own code, or LlamaIndex or LangChain
What you need	No code	A few lines of code	No code, but a server to run	Programming
Where documents go	The vendor	The vendor	Your server, but chunks still go to the model	Wherever you decide
Control over retrieval	None	A few settings	Some, depending on the product	Full
Good for	Personal use with a few public documents	Adding Q&A to an existing app	Sensitive documents, for a group with IT support	Teaching, research, and systems you keep testing

NotebookLM needs no code, but you give up control and upload your documents to Google https://notebook.google.com/

1. Create a notebook
2. Add the handbook PDF and the tuition web page
3. Ask the five questions

Each answer comes with citations that you can click to see the chunk it came from.

What it does for you
It converts the files, splits them, searches them, and writes the prompt.

What you give up
You cannot change any of those steps, and your documents are stored by Google.

Your turn Watch now. You can build your own after class (Notebook 1, Section 2).

Instructor Chrome → NotebookLM tab → ask Q2 and Q3 live

There are four main ways to build a RAG assistant, from no code to fully custom

	1. Consumer platform	2. Managed RAG API	3. Self-hosted application	4. Built from scratch
Examples	NotebookLM, Claude Projects	Gemini File Search, OpenAI File Search	AnythingLLM, Open WebUI, Dify	Your own code, or LlamaIndex or LangChain
What you need	No code	A few lines of code	No code, but a server to run	Programming
Where documents go	The vendor	The vendor	Your server, but chunks still go to the model	Wherever you decide
Control over retrieval	None	A few settings	Some, depending on the product	Full
Good for	Personal use with a few public documents	Adding Q&A to an existing app	Sensitive documents, for a group with IT support	Teaching, research, and systems you keep testing

Gemini File Search adds document Q&A to a program with just three API calls

```
# 1. Create a document store
store = client.file_search_stores.create(
    config={"display_name": "edrag"})

# 2. Upload a file. Google converts, splits, and indexes it.
client.file_search_stores.upload_to_file_search_store(
    file="ITM_Student_Handbook.pdf",
    file_search_store_name=store.name)

# 3. Ask a question with the File Search tool turned on
result = client.interactions.create(
    model="gemini-3.5-flash-lite", input=question,
    tools=[{"type": "file_search",
            "file_search_store_names": [store.name]}])
```

- There is no server to set up or maintain.
- Google stores and processes your files.
- Each answer cites the file and page it came from.

Your turn Notebook 1 → Section 3.3: run it, then score Q1–Q5 in the next cell.

Instructor No key? Press Enter in 3.1, and the gateway uses the instructor's store.

A managed API lets you adjust a few settings, but not how it reads or searches your documents

What you can change

- The prompt, by adding instructions to the question
- The chunk size and overlap
- The embedding model
- The model that writes the answer
- Which files to search, using metadata

What you cannot change

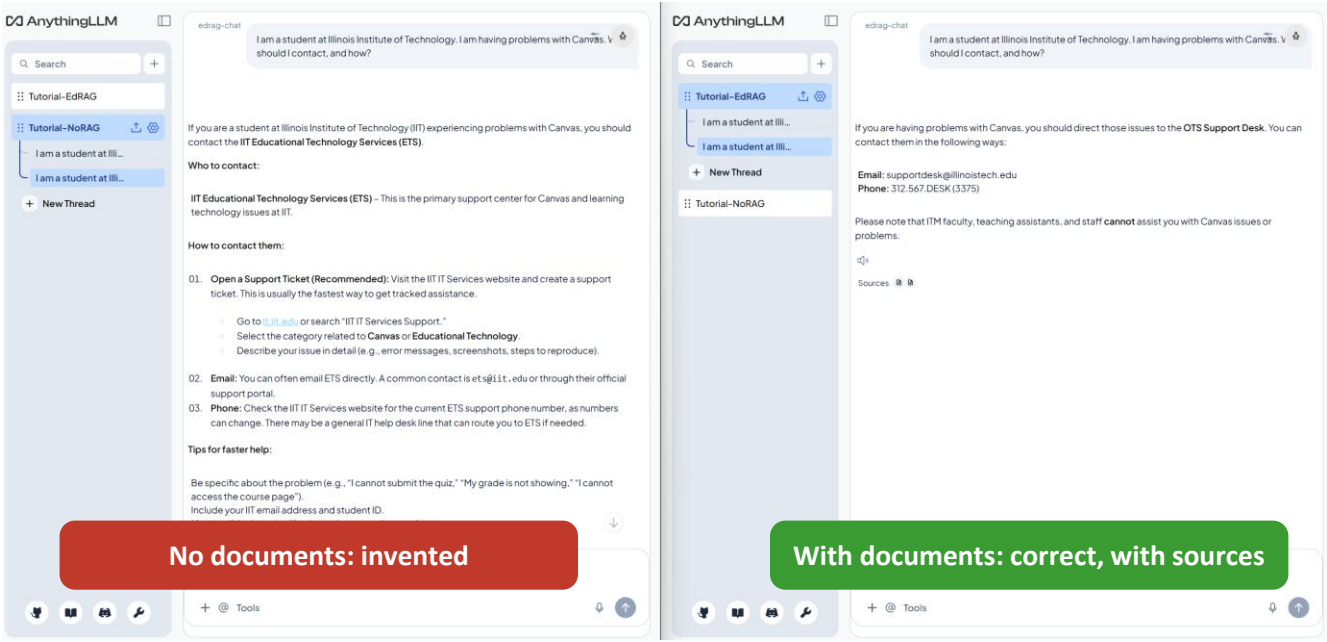
- How PDF and HTML files are turned into text
- Splitting by heading, or adding section paths
- How many chunks are used, and how they are ranked
- Seeing the retrieved chunks, beyond the citations

Example: the handbook says ITM faculty and staff **cannot assist** with Canvas; File Search wrote that they are **typically unable** to assist. We cannot see why.

There are four main ways to build a RAG assistant, from no code to fully custom

	1. Consumer platform	2. Managed RAG API	3. Self-hosted application	4. Built from scratch
Examples	NotebookLM, Claude Projects	Gemini File Search, OpenAI File Search	AnythingLLM, Open WebUI, Dify	Your own code, or LlamaIndex or LangChain
What you need	No code	A few lines of code	No code, but a server to run	Programming
Where documents go	The vendor	The vendor	Your server, but chunks still go to the model	Wherever you decide
Control over retrieval	None	A few settings	Some, depending on the product	Full
Good for	Personal use with a few public documents	Adding Q&A to an existing app	Sensitive documents, for a group with IT support	Teaching, research, and systems you keep testing

AnythingLLM runs on our own server, and with our documents it gives the correct contact



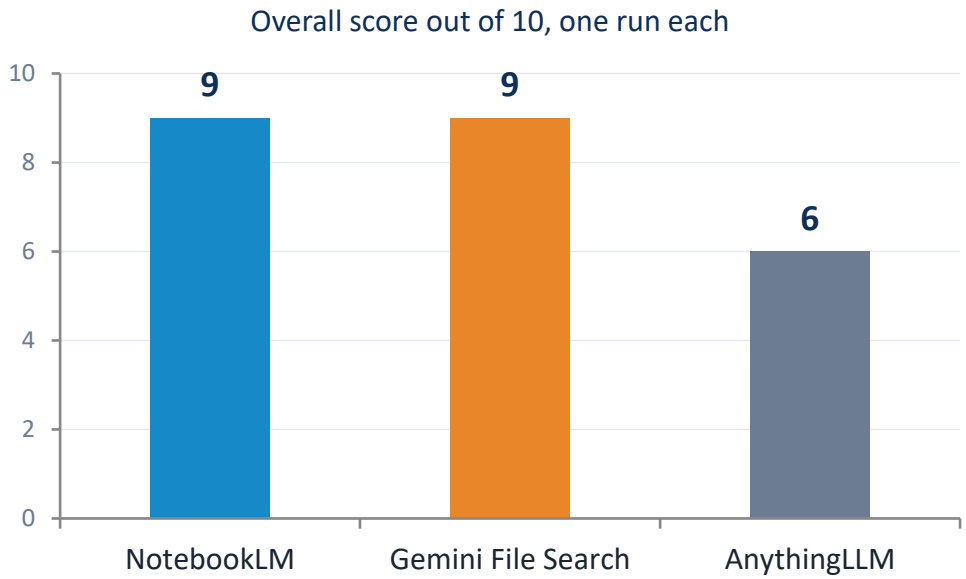
Same model, same question.

Our documents stay on our server. The chunks it retrieves go to Qwen3.8-27B on Jetstream2, which also runs inside Indiana University.

Your turn Notebook 1 → Section 4: run 4.1 and 4.2, then score Q1–Q5.

Instructor (optional) Chrome tab `localhost:3001`: ask the same question live in both workspaces

NotebookLM and File Search scored 9 out of 10, and AnythingLLM scored 6 because of how it read the web page



NotebookLM and File Search each lost one point on Q2, for different reasons.

AnythingLLM got both tuition questions wrong. It split the raw web page, code and all, so the tuition table never reached the model.

It uses the same model as our own assistant in Module 2. The difference comes from how the documents were prepared.

Your turn Notebook 1 → Section 5: `compare_overall()` draws a chart of your own scores.

AGENDA

Module 2 (35 minutes): you build your own advising assistant, one step at a time



minutes

- Opening · 5 min Log in and check that everything works
- Module 1 · 20 min Four Ways to Build RAG
- **Module 2 · 35 min Building an Advising Assistant from Scratch**
- Module 3 · 8 min How Far Can We Trust the Answers?
- Module 4 · 7 min Optional: Course Advising as a More Complicated Task, then Q&A

You will build the assistant in five steps, comparing a simple way and a better way in each

1. **Convert** plain text vs. Markdown
2. **Chunk** fixed length vs. by heading
3. **Embed and retrieve** turn chunks into vectors, then find the closest ones
4. **Generate** a plain prompt vs. a grounded prompt
5. **Evaluate** score Q1–Q5 and compare all four approaches

In every step, you first make a guess, then run both ways, then change one setting yourself. No programming is needed: you change a number or a sentence. “Core idea” boxes point out the few lines of code worth remembering.

Your turn Open `02_rag_from_scratch.ipynb` and run Section 0.

Instructor Run each cell together with the class, and pause at every Guess.

You will build the assistant in five steps, comparing a simple way and a better way in each

1

Convert
plain text vs. Markdown

2

Chunk
fixed length vs. by heading

3

Embed and retrieve
turn chunks into vectors, then find the closest ones

4

Generate
a plain prompt vs. a grounded prompt

5

Evaluate
score Q1–Q5 and compare all four approaches

Plain-text extraction keeps the words but loses the structure of the table

Plain text (BeautifulSoup)

```
Living on Campus
Living Off Campus
Living at home
Tuition
(billable expense)
$53,198
$53,198
$53,198
Fees
(billable expense)
$1,850
…
```

Markdown (html-to-markdown)

```
|          | On Campus | Off Campus | At home |
| -------- | --------- | ---------- | ------- |
| Tuition  | $53,198   | $53,198    | $53,198 |
| Fees     | $1,850    | $1,850     | $1,850  |
| Housing  | $9,776    | $11,700    | $4,950  |
| Food     | $8,772    | $4,950     |         |

Row labels shortened to fit the slide.
```

In plain text, the model has to guess which number belongs to which row and which column. In Markdown, each row keeps its label.

Your turn Notebook 2 → Section 1.1: run the cell and look at the table in plain text.

Markdown keeps headings and tables as plain text, which is why many RAG systems convert documents to it first

- It keeps the structure of a document, such as headings, lists, and tables, with very little extra markup.
- Language models read it well, because they saw a lot of Markdown during training.
- It uses far fewer tokens than HTML, which also carries tags, menus, and scripts.
- Its headings make it easy to split a document into sections later.

File type	Common converters
HTML	html-to-markdown (used today), MarkItDown, Pandoc
Word (.docx)	Pandoc, MarkItDown, Mammoth
PDF	Docling (used today), Marker, pymupdf4llm, MarkItDown (text only)

No converter handled the merged cells correctly, so we label them before converting

The undergraduate table has one cell, $4,950, that covers both Housing and Food for students living at home.

Approach	What it did with the merged cell	What goes wrong
Docling	Left the Food cell empty in one table, and copied the value in another	Food reads as $0, or the column adds up to too much
pandas.read_html	Copied $4,950 into both rows	The column adds up to more than its total
html-to-markdown	Left the Food cell empty	The model may read the food cost as $0
Label first, then convert	Wrote "$4,950 (combined: Housing and Food)" and "(included in Housing)"	**Every column adds up, and no cell is empty**

Core idea When a general tool gets your data wrong, a few lines based on knowing your data can fix it: `cell.string = f"{value} (combined: …)"`



Your turn Section 1.2: pick your guess first, then run the cells.

You will build the assistant in five steps, comparing a simple way and a better way in each

1

Convert
plain text vs. Markdown

2

Chunk
fixed length vs. by heading

3

Embed and retrieve
turn chunks into vectors, then find the closest ones

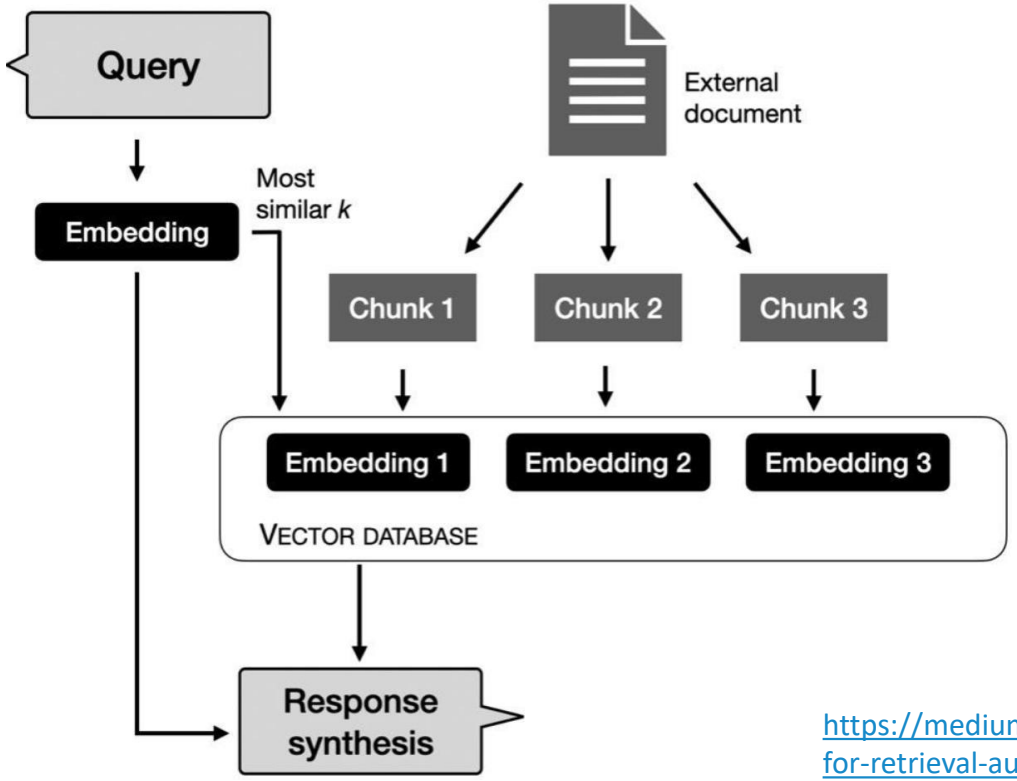
4

Generate
a plain prompt vs. a grounded prompt

5

Evaluate
score Q1–Q5 and compare all four approaches

Pipeline: Chunking → Embedding → Retrieval



https://medium.com/@sahin.samia/mastering-chunking-strategies-for-retrieval-augmented-generation-rag-c0d3d09f80f2

Fixed-length chunks can cut a number away from the heading that explains it

Fixed length, 300 characters

```
…er year. Some programs require additional
credit hours beyond the standard 18 credit
hours per year. Please confirm your program
requirements and calculate your tuition
accordingly.

| Tuition (billable expense) | $34,650 |
```

Which students is this $34,650 for? The chunk does not say.

By heading, with the section path

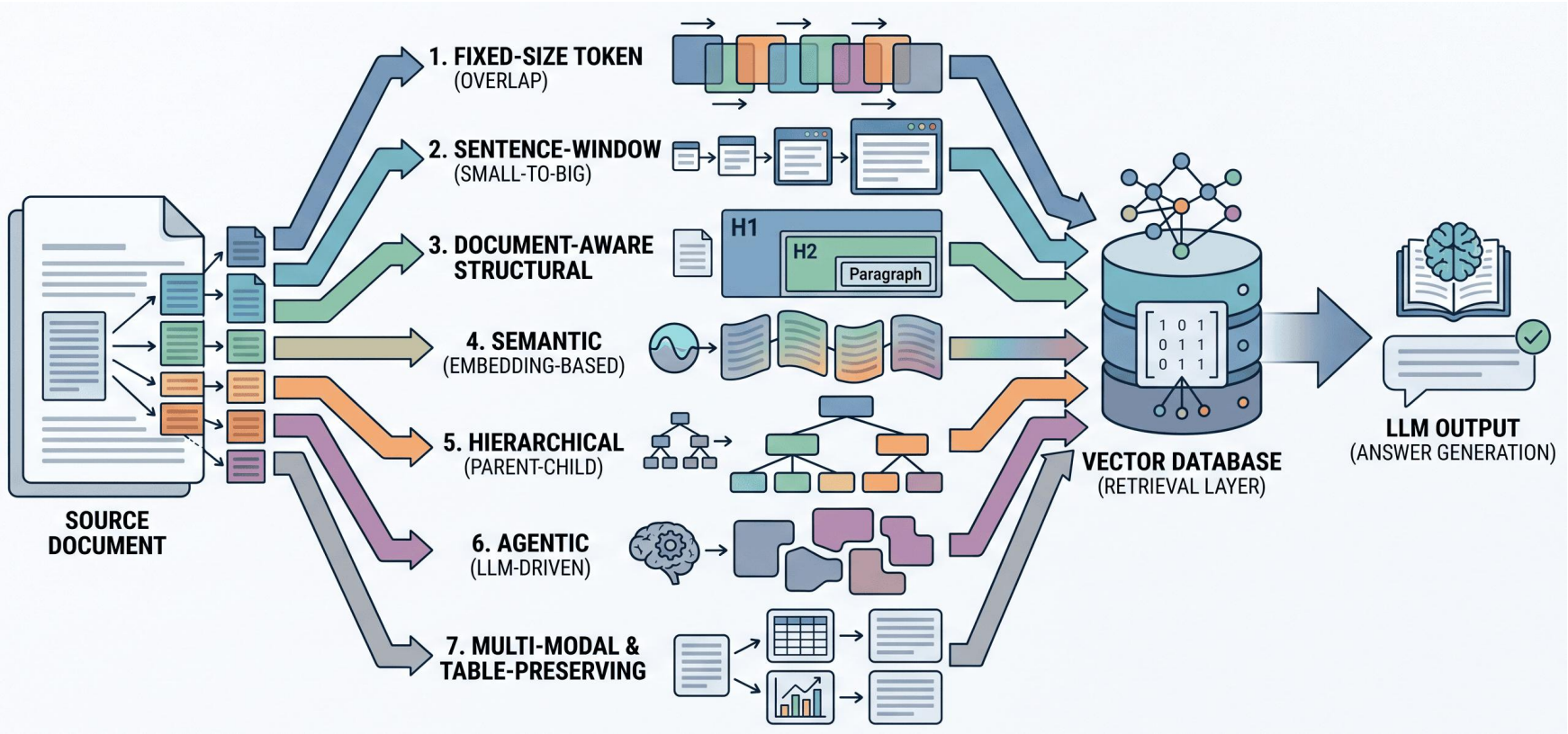
```
Tuition, Fees, and Costs >
  New Graduate Student (Mies Campus)

Estimates for the 2026–2027 academic
year are listed below. These estimates
are based on 18 credit hours per year. …
| Tuition (billable expense) | $34,650 |
```

Core idea start every chunk with its section path: `f"{path}\n\n{piece}"`

Your turn Section 2: guess, run, then set `SIZE` to 300 and to 2000.

There are several chunking strategies



https://machinelearningmastery.com/7-chunking-strategies-that-decide-whether-your-rag-works/

An embedding turns each chunk into 384 numbers, so texts with similar meaning get similar numbers

- We use **bge-small-en-v1.5**, a small open model that runs well on a regular CPU.
- Other models use 768 to 3,072 numbers. More numbers take more storage and computing, but accuracy depends mostly on how the model was trained.
- Questions and chunks must be turned into numbers by the same model.
- bge-small reads English only, and at most 512 tokens per chunk.

Preset C

122 chunks

× 384 numbers each

= one matrix E, with one row per chunk

Retrieval is one matrix multiplication that scores every chunk against the question *q* at once

```
scores = E @ q
(122 × 384) times (384)  →  122 scores
```

- Every vector has length 1, so the product is the similarity between the question and each chunk.
- We sort the scores and keep the top k chunks.

What we saw on the tutorial server

- For Q3, the top chunk was the Canvas section, with a score of 0.709. The next one scored only 0.550.
- Leaving out bge's query prefix barely changed the ranking for Q4: the top chunk scored 0.748 with it and 0.750 without.
- With the top 5 chunks, all three presets found the answers to Q1–Q4.

Your turn Section 3: run the cells, then set `K` to 3 and to 8 and compare.

You will build the assistant in five steps, comparing a simple way and a better way in each

1

Convert
plain text vs. Markdown

2

Chunk
fixed length vs. by heading

3

Embed and retrieve
turn chunks into vectors, then find the closest ones

4

Generate
a plain prompt vs. a grounded prompt

5

Evaluate
score Q1–Q5 and compare all four approaches

A strict prompt stopped the guessing, but it also made the model hold back the tuition figures

Plain prompt

“Answer the student’s question. Here is some information that may help.”

Grounded prompt: four rules

1. Use only the chunks; do not add facts from memory.
2. Cite the chunks after each sentence, such as [1].
3. If the chunks do not answer the question, say so, then mention related information.
4. If the answer depends on the student’s situation, answer for each case.

Result for Q2: in 5 out of 5 runs, the grounded prompt made the model say it could not find an international rate, and stop there. It never listed the three tuition rates that the retrieved chunks did contain. (The notebook’s prompt calls the chunks “passages.”)

Your turn Section 4: run the Q2 and Q5 cells and compare the two prompts.

Changing one sentence fixed Q2, and the other four questions stayed correct

New sentence for rule 3: *"Then give everything the chunks do say that could help; if it differs for different groups of students, such as program level or visa status, give it for each group."*

Prompt	Q1	Q2	Q3	Q4	Q5	Total
Plain	5/5	5/5	5/5	5/5	5/5	25/25
Grounded	5/5	**0/5**	5/5	5/5	5/5	**20/25**
Grounded, one sentence changed	5/5	5/5	5/5	5/5	5/5	25/25

Runs passing an automatic key-fact check, Qwen3.8-27B, 5 runs per question

Instructor Show the check table from Section 4 (3 runs each). Students can skip running it.

Two cautions

The check only looks for key facts. One plain-prompt answer to Q2 also guessed that U.S. universities usually charge everyone the same tuition. A person would score it 0.

With other models, the same change helped less. Test again when you change the model.

With the same documents and questions, our own assistant scored 10 out of 10

Approach	Q1	Q2	Q3	Q4	Q5	Total
NotebookLM	2	1	2	2	2	**9**
Gemini File Search	2	1	2	2	2	**9**
AnythingLLM	0	0	2	2	2	**6**
Built from scratch	2	2	2	2	2	**10**

AnythingLLM and our assistant used the same model, Qwen3.8-27B. The difference comes from how we prepared the documents.

Read this with care

Each approach was run once, and answers change from run to run. This table shows typical behavior on five questions, not a final ranking.



Your turn Section 5: score your assistant, then run `compare_overall()` to see all four approaches.

We checked every answer by hand; other evaluation methods can run faster or at a larger scale

Human inspection (used today)

People read each answer against the expected answer and a scoring rule.

Most trusted, but slow; scorers need clear rules to agree.

Key-fact checks

A script checks that each answer contains the facts it must have, as in Notebook 2, step 4.

Fast and repeatable, but misses anything wrong that was added.

Retrieval metrics

Check whether the chunk with the answer is in the top k (hit rate, recall@k, MRR).

Pinpoints retrieval problems, but needs the right chunk marked for each question.

LLM as a judge

Another model rates whether each answer sticks to its chunks and answers the question (for example, RAGAS or DeepEval).

Scales to many questions, but the judge can be wrong too.

Repeated runs and regression tests

Ask each question several times, and rerun the whole test set after every change.

Shows how stable the answers are and catches new mistakes.

Feedback after launch

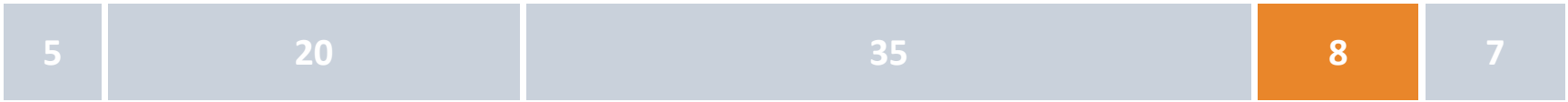
Collect ratings from users, and review logged questions and answers.

Shows real use, but problems appear only after students see them.

In practice, combine them: run automatic checks after every change, and have people review a sample and every high-stakes answer.

AGENDA

Module 3 (8 minutes): even a grounded assistant can be wrong, so we design it to be checked



minutes

- Opening · 5 min Log in and check that everything works
- Module 1 · 20 min Four Ways to Build RAG
- Module 2 · 35 min Building an Advising Assistant from Scratch
- **Module 3 · 8 min How Far Can We Trust the Answers?**
- Module 4 · 7 min Optional: Course Advising as a More Complicated Task, then Q&A

A RAG answer can still go wrong at every step, and we saw each case in our own tests

Documents: The documents themselves can be wrong or inconsistent: the catalog lists ITM 691 in a table but ITMT 691 in a footnote.

Conversion: A merged table cell became an empty Food cell, which reads as $0.

Chunking: With 300-character chunks, $34,650 lost the heading that says who it is for.

Retrieval: A keyword search for “F-1” missed the rule, because the handbook spells it “Fl Visa.”

Generation: Even with the right chunks, the grounded prompt held back the three tuition rates in 5 of 5 runs.

Run-to-run changes: The same question gave different answers from one run to the next, even at temperature 0.

RAG reduces mistakes and makes them easier to check. It does not guarantee a correct answer, so design for checking, not for blind trust.

Showing the source chunks with every answer lets students check it themselves

If the answer can be wrong, why not just show students the documents?



More reading for students

Nothing to check against (avoid)

- Showing only the documents does not solve the problem: retrieval can still miss, and questions like Q2 and Q4 need several chunks combined.
- An answer with citations is quick to read and still easy to check.
- Match the design to the risk: answer simple questions directly with citations; for visas or eligibility, quote the rule and suggest confirming with an advisor.

A confidence signal can tell students when to check the source, but it must be tested first

The model rates itself
Easiest to add, but least reliable: models are often confident when they are wrong.

Retrieval score
A low score means no relevant chunk was found, but a high score does not prove the answer is right.

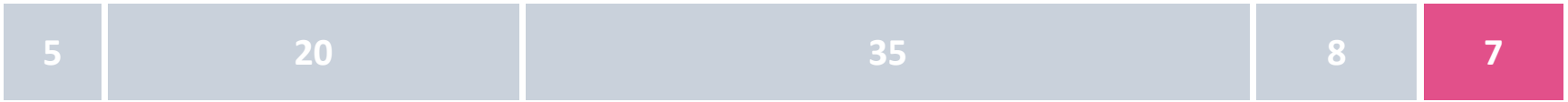
Agreement across runs
Ask the same question several times; disagreement is a warning sign. It costs more model calls.

Sentence-by-sentence check
A second model checks each sentence against its chunk, which catches invented facts.

- Before trusting the signal, test it on questions with known answers: “high” should really mean mostly correct.
- Show High, Medium, or Low rather than a percentage. When it is low, show the source and suggest contacting an advisor.

AGENDA

Module 4 (7 minutes, optional): course advising needs rules and student data, not just retrieval



minutes

- Opening · 5 min Log in and check that everything works
- Module 1 · 20 min Four Ways to Build RAG
- Module 2 · 35 min Building an Advising Assistant from Scratch
- Module 3 · 8 min How Far Can We Trust the Answers?
- **Module 4 · 7 min Optional: Course Advising as a More Complicated Task, then Q&A**

Recommending courses means applying rules to a student's record, not finding a similar paragraph

"I am starting the ITM PhD with an ITM master's degree. Which courses should I take this spring?"

- **Student record:** courses taken and grades
- **Degree rules:** required and elective courses, which depend on the student's background
- **Prerequisites:** which courses must come first, sometimes with a minimum grade
- **Schedule:** which courses are offered this term

→

Courses this student can and should take

This answer comes from combining all four inputs with logic. Retrieval can find the rules, but it cannot do this combining.

A hybrid design lets rules decide what is allowed, while RAG explains the policies



- Rules, not the language model, decide eligibility, so the same record always gives the same result.
- The language model explains the result in plain words and quotes the policy that applies.
- An advisor stays in the loop for the final decision.

Using an advising assistant responsibly requires people in the loop and well-protected student data



Advisors make the final decisions; the assistant helps them prepare.



Policy documents have an owner and a regular update schedule.



Student records are protected under FERPA, with access limited to the right people.



Questions and answers are logged so problems can be found and fixed.



Uncertain or high-stakes questions are passed to a person.



Free AI tools may use what you enter to improve their products; use them only with public documents.

Thank you!

Questions and discussion

Dr. Yong Zheng · yzheng66@illinoistech.edu

Materials: academictutorials.github.io/EdRAG

This work used Jetstream2 at Indiana University through allocation CIS260631 from the ACCESS program, supported by NSF grants #2138259, #2138286, #2138307, #2137603, and #2138296. We thank Cohere and ThetaCloud for API credits.





ILLINOIS TECH